Code review — August 2026

A full defect-first review of the Python desktop application, covering the inference pipeline, the model wrappers, the export subsystem, configuration and caching, the Qt user interface, and the test suite and tooling.

Three critical defects found by this review have been fixed and are recorded in Fixed in this pass. Everything under Outstanding findings is still open and is written to be used as a working backlog.

Verdict

The codebase is in good shape. The layering is clean — `audio` feeds `pipeline`, which drives `models`, which produces results for `export`, with `ui` sitting on top and no upward dependencies. The test suite is fast, hermetic, and mostly asserts real behaviour rather than the absence of crashes.

Several things that are commonly got wrong in an application like this are got right here. All HTML written by the WeasyPrint exporter is escaped at the point of interpolation. The Hugging Face token goes to the operating system keyring rather than into a settings file. Transcript and speaker-name caches are written through a temporary file and `os.replace`, so they cannot be torn in half by a crash. No subprocess is invoked with `shell=True`, and FFmpeg arguments are passed as an argument vector rather than a command string. Every file in the export and cache layers is opened with an explicit `encoding="utf-8"`, which matters on Windows where the default is cp1252 and would otherwise corrupt any non-ASCII transcript.

The problems are concentrated in three areas: silent data loss in transcript assembly, worker lifecycle management in the Qt layer, and the near-total absence of project tooling.

Metrics

Measure	Value
Files tracked in git	241
Production modules under `speaker_transcriber/`	71
Test files / test functions / collected items	33 / 361 / 666
Test suite runtime	~20 seconds, fully offline
Largest module	`speaker_transcriber/ui/main_window.py`, 2,650 lines
Linting, formatting, type checking, CI	none configured

The five largest modules are `ui/main_window.py` (2,650), `export/pdf_exporter.py` (1,086), `models/summarization.py` (892), `prompts/__init__.py` (601), and `models/notes_assembly.py` (561).

Method

Each subsystem was read in full rather than sampled. Every finding recorded at Critical or High severity was then verified directly against the source, and the three critical defects were confirmed by executing the pre-fix code to observe the failure. Findings that could not be reproduced or that did not survive verification were discarded — one reported defect claiming that Qt signal connections accumulate across runs was withdrawn after inspection showed each run constructs a fresh worker object, so its connections are its own. The real defect in that code is a leak, and it is recorded as such under Medium.

Fixed in this pass

Transcript segments were silently dropped

`words_from_segments` in `speaker_transcriber/pipeline/speaker_assignment.py` tested `if segment.words:` and, when that list was non-empty, iterated it while discarding any entry that lacked usable text or timestamps. If no entry survived, the `elif segment.text:` fallback was unreachable and the entire segment vanished from the transcript with no error and no log line.

This was reachable in practice. `models/alignment.py` builds each word as `str(word.get("word", "")).strip()`, so WhisperX returning a segment whose words are all blank-but-timestamped produced exactly this shape. Running the pre-fix function against such a segment returned an empty list where it should have returned the spoken text.

The fix collects usable words into a local list and falls back to the segment text whenever that list ends up empty, so speech is preserved as a single uncertain block rather than lost.

Closing during PDF export could destroy the window with a live thread

`PdfExportWorker` was the only worker without a `request_cancel()`. During out-of-memory recovery it blocks on an unbounded `self._recovery_event.wait()` in `NotesMemoryRecoveryMixin._apply_recovery`, and `MainWindow.closeEvent` never unblocked it — it waited five seconds, timed out, and accepted the close while the thread was still running. `SummarizationWorker` already did this correctly by calling `provide_recovery(STOP)` from its `request_cancel()`.

`PdfExportWorker` now carries a `cancel_event`, exposes a `request_cancel()` that mirrors the summarization worker, passes the event into both `RequirementsSummarizer` constructions, and checks it before writing the PDF. `closeEvent` calls it before waiting.

Note that PDF export still has no user-facing Cancel button; this fix addresses the shutdown hang, not the missing control. See No user-facing cancel for PDF export.

Single-file caches collided across directories

`MediaSource.cache_key()` returned the bare filename stem for single-file sources, so two recordings named `meeting.mp4` in different folders shared one cache entry, and replacing a recording in place kept serving the stale transcript. The multi-file branch immediately below already fingerprinted path, modification time, and size.

The key is now fingerprinted for single files too, through a shared `_fingerprint` helper. The helper produces byte-identical output to the previous multi-file code when every file is present, so existing multi-file caches continue to resolve, and it degrades to a path-only key instead of raising when a recording has been moved or deleted.

Because this would otherwise orphan every existing single-file cache, `TranscriptCache` adopts legacy files on first access: it renames the old stem-named transcript, speakers, and summary files onto the new key, but only after confirming that the cached transcript's recorded `source_file` resolves to the same recording. A legacy cache belonging to a different file of the same name is deliberately left untouched, which is precisely the collision the change exists to prevent.

Test changes that accompanied the fixes

Three PDF tests construct workers through `PdfExportWorker.__new__` and wire attributes by hand, bypassing the constructor, so each needed `cancel_event` added. Two cache tests asserted exact filenames such as `meeting.json`; they now assert the stem prefix and the extension, preserving their intent — the cache file is named after the recording rather than its folder — without pinning the colliding format.

Nine regression tests were added: three covering segment survival in `tests/test_speaker_assignment.py`, four covering cache identity and legacy adoption in `tests/test_transcript_cache.py`, and two covering PDF cancellation in `tests/test_notes_oom_recovery.py`, including the shutdown case where the user never answers the out-of-memory prompt.

Outstanding findings

High

Underscores in inline code abort PDF export

Found while rendering this document to PDF with the project's own helper. `markdown_to_reportlab` in `speaker_transcriber/export/pdf_exporter.py` converts backtick spans to `<font name="Courier">` first and only then applies the emphasis patterns, over the whole string including the code it just wrapped. `_ITALIC` treats a lone _ as an emphasis delimiter, so an underscore inside one code span pairs with an underscore inside the next:

```
in   See `main_window.py` and `pdf_exporter.py` here.
out  See <font name="Courier">main<i>window.py</font> and
     <font name="Courier">pdf</i>exporter.py</font> here.
```

The `<i>` tags interleave with the `<font>` tags, ReportLab's parser raises `ValueError: Parse error: saw </font> instead of expected </i>`, and the export fails outright. Two snake_case identifiers in one paragraph is enough, which is unremarkable in notes for a technical meeting.

The same ordering silently corrupts output short of crashing. A lone dunder filename in a code span comes out as bold markup with the underscores eaten, rendering `__init__.py` as a bolded `init.py`, and in plain prose the phrase `main_window.py and pdf_exporter.py` loses both pairs of underscores to italics.

Set code spans aside before applying emphasis and restore them afterwards, and consider dropping _ as an italic delimiter altogether given how much of this codebase's vocabulary is snake_case. `scripts/render_doc_pdf.py` has a worked version of the placeholder approach.

Settings can be wiped without warning

`SettingsStore.save` in `speaker_transcriber/config.py` uses `write_text`, which truncates before writing, so an interrupted save leaves partial JSON on disk. `SettingsStore.load` then catches `OSError`, `ValueError`, `TypeError`, and `JSONDecodeError` together and returns `AppSettings()` with no logging, so the user silently loses every setting on the next launch and has no way to tell why.

The cache layer already has the pattern to copy. Write through `NamedTemporaryFile` and `os.replace`, log the failure at warning level, and consider preserving the unreadable file as `settings.json.broken` rather than discarding it.

CSV formula injection

`render_csv` in `speaker_transcriber/export/csv_exporter.py` writes the speaker name and segment text into cells without sanitisation:

speaker_transcriber/export/csv_exporter.py — lines 16–23

```
        writer.writerow(
            [
                f"{segment.start:.3f}",
                f"{segment.end:.3f}",
                display_speaker(result, segment.speaker),
                segment.text,
            ]
        )
```

A cell whose value begins with =, +, -, or @ is interpreted as a formula by Excel and Google Sheets. Since both fields derive from transcribed speech or user-entered speaker names, a crafted or merely unlucky value executes on open. Prefix at-risk cells or route every string field through a shared sanitiser.

`speechbrain` is missing from `pyproject.toml`

It is imported at runtime by `speechbrain_compat.py`, `models/diarization.py`, `ui/worker.py`, and `app.py`, and it is listed in `requirements.txt`, but it does not appear in the `dependencies` array of `pyproject.toml`. A `pip install -e .` therefore produces an installation that fails at diarization time. The two dependency files otherwise agree.

A real meeting export is committed to the repository

`Standard recording 53_meeting.pdf` is tracked in git and was modified again during this review, growing from 13,478 to 18,249 bytes. Beyond being build debris in a source tree, it is a generated transcript of an actual meeting, so removing it warrants a deliberate decision about the existing history rather than a simple `git rm`.

Subtitle exports break on multi-line text

Both renderers in `speaker_transcriber/export/subtitle_exporter.py` interpolate `segment.text` directly into a line-oriented format:

speaker_transcriber/export/subtitle_exporter.py — lines 12–21

```
        cues.append(
            "\n".join(
                [
                    str(index),
                    f"{subtitle_timestamp(segment.start)} --> "
                    f"{subtitle_timestamp(segment.end)}",
                    f"{display_speaker(result, segment.speaker)}: {segment.text}",
                ]
            )
        )
```

A newline inside `segment.text` inserts a blank line into the cue, which terminates it early and desynchronises every subsequent cue index. The WebVTT renderer has the same problem plus an unescaped `<v ...>` voice tag, so a speaker name or transcript fragment containing <, >, or `-->` corrupts the cue structure. Normalise newlines and escape the markup.

Diarization cannot be cancelled

`models/diarization.py` checks `cancel_event` only after the pyannote pipeline call returns, so cancelling a long recording has no effect until the whole file finishes. The retry loop in `_load_pipeline` compounds this by sleeping up to 30 seconds per attempt without checking the event. Alignment has the same shape in a milder form. Poll the event during the operation and in the backoff sleep.

FFmpeg calls have no timeout

The `subprocess.run` calls for `ffprobe` and for concatenation in `speaker_transcriber/audio/ffmpeg.py` (lines 92 and 234) pass no `timeout`, and the progress-reading loop around `Popen` for extraction has no deadline. A corrupt or stalled input hangs the call indefinitely. Derive a timeout from the media duration and raise `MediaError` when it expires.

Blocking work on the GUI thread

`_refresh_resource_meters` runs on a one-second timer and calls `query_gpu_stats()`, which shells out to `nvidia-smi` with a 1.5-second timeout, so the event loop can stall for well over a second every second on machines where that tool is slow. Separately, `_cap_ctx_slider_for_model` performs a synchronous Ollama `show()` when the model combo changes. Both belong on a worker thread that emits results as signals.

Lower-impact instances of the same problem: transcript cache loads and saves, `export_result()`, and the Hugging Face cache directory walk in the model combo population all run on the main thread.

`closeEvent` cleans up only three of five workers

`MainWindow.closeEvent` handles `worker`, `summary_worker`, and `pdf_export_worker`, but leaves `duration_probe_worker` and `ollama_model_worker` running, and stops neither `gpu_stats_timer` nor `log_timer`. Both timers keep firing into a window that is being torn down. `AddModelsDialog._stop_worker` is a good model for what this should look like.

Related: the duration probe's cancellation is ineffective, because `MediaDurationProbeWorker.run` never checks `isInterruptionRequested()`, and a replacement probe is connected without disconnecting the previous one, so rapid file additions can deliver a stale duration.

Medium

Finished workers accumulate on the main window

Every job constructs a worker parented to the window — `ProcessingWorker([...], options, self)` — and never calls `deleteLater()`. The finished `QThread` therefore survives as a child of `MainWindow`, holding its options and results, for the lifetime of the process.

This is a leak and nothing more. Because each run creates a new object, its signal connections belong to it alone, and a finished thread does not emit again, so slots are not invoked more than once.

No user-facing cancel for PDF export

Summarization has a Cancel button; PDF export has none. The worker now supports cancellation, so this is a matter of wiring a button to `request_cancel()`.

PDF export failure leaves the summary tab mid-render

Streaming export sets the summary view to `"Generating summary…"`. The failure and cancellation handlers clear `_pdf_streaming_summary` but never restore the previous markdown, so the view stays stuck on that placeholder. `_summary_cancelled` already does the right thing and can be mirrored.

Relatedly, `_summary_failed` writes to the log pane without showing a modal, unlike the transcription and PDF failure paths, so a summarization failure is invisible when the log section is collapsed.

Models are reloaded on every run

Whisper, the WhisperX alignment model, and the pyannote pipeline are each constructed fresh per invocation, with nothing cached in `ModelManager`. On short files and batch runs, load time dominates. Caching by `(model_id, device, compute_type)` would help, though it trades against the one-model-resident-in-VRAM design described in the pipeline documentation and should be weighed against it.

Speaker assignment is quadratic

`assign_speakers` calls `assign_word` per word, and each call scans every diarization segment. A long meeting — roughly 30,000 words against 2,000 speaker turns — costs about 60 million interval comparisons. Sorting the turns once and querying an interval index would make this linear in practice.

Errors are flattened into generic failures

`models/summarization.py` catches broadly and re-raises a plain `RuntimeError`, discarding the original type, so the UI cannot distinguish an out-of-memory condition from a refused connection or an empty response. `pipeline/processor.py` treats every alignment exception as "continue without alignment", which turns genuine misconfiguration into silently degraded output. `models/hf_catalog.py` returns an empty list when hub listing fails, so being offline is indistinguishable from having no models. In each case, re-raise known types unchanged and reserve the broad catch for genuinely unknown failures.

`main_window.py` is a god object

At 2,650 lines it carries at least eleven distinct responsibilities: the menu and recent-files list, widget construction, model combo population for three model families, resource meters, source input and drag-and-drop, the transcription lifecycle, the speaker table and its navigation, Ollama settings, export, the

summary and PDF flow including out-of-memory recovery, and window persistence. The summary/PDF/OOM block and the model-catalog block are the two largest and most self-contained extraction candidates.

Correctness details in merging and assignment

`transcript_merger.merge_words` clamps a negative gap to zero, so words whose timestamps overlap are merged as though adjacent, and it assumes its input is sorted by start time without enforcing it. In `assign_word`, the containment test uses `segment.start <= midpoint <= segment.end` at both ends, so two adjacent turns sharing a boundary can both contain a midpoint and the tie is broken by list order. Half-open intervals would resolve it.

Cache and mutation details

`TranscriptCache.save` assigns to `result.speakers` on the caller's object, so callers holding that result observe the names change underneath them. `load_summary` returns `None` on the first `OSError` instead of continuing to the legacy path, so a transient error on the new filename hides a valid older summary. The ReportLab exporter configures page chrome by assigning to `_NumberedCanvas` class attributes, which would interleave badly if two exports ever ran concurrently. Neither the transcript cache nor the debug log has any size or age bound.

No linting, formatting, type checking, or CI

There is no ruff, flake8, black, isort, mypy, pyright, pre-commit, or CI configuration anywhere in the repository; pytest configuration in `pyproject.toml` is the whole of the tooling. Given a suite of 666 tests that runs offline in twenty seconds, a CI workflow that simply runs it is close to free and is the highest-leverage single addition available.

Note that `pytest-timeout` is declared nowhere but is not installed either, so `--timeout` is unavailable; and `pytest-mock` is declared in the dev extras but never used, as every test mocks through `monkeypatch`.

Low

- **Untested modules.** Twenty-four production modules have no direct test coverage. The three that matter are `models/transcription.py`, `models/alignment.py`, and `models/diarization.py` — the core inference wrappers, and the neighbourhood of the segment-loss defect fixed above. Most of the untested remainder is Qt widgets and compatibility shims.
- **Loose scripts in the repository root.** `scratch_smoke.py`, `scratch_ui_check.py`, `smoke_pdf_dialog.py`, `smoke_theme_grid.py`, `tmp_ollama_probe.py`, and `_preview_names.py` are throwaway — two say so in their own docstrings, and two write PNGs into the root. A `scripts/` directory already exists for the ones worth keeping. `app.py` and `summarize_gui.py` are legitimate entry points and should stay.
- `debug_log.py` **is development instrumentation.** It writes to a repo-relative `debug-4ba2aa.log` with a hard-coded session identifier. It should be removed or gated behind an explicit debug flag and pointed at `app_data_dir()`.
- **Markdown export does not escape structure.** Speaker names and transcript text are interpolated into headings, so a newline in a speaker name produces spurious headings and text beginning with # or | alters the document structure.

- **Duplication across exporters.** `_TurnColors` is duplicated verbatim between the ReportLab and WeasyPrint exporters; inline-markup handling exists in three variants that have already drifted, with ReportLab converting `**bold**` and WeasyPrint not; speaker-name persistence is implemented twice. The individual exporters also assume their parent directory exists, which only holds because `export_result()` creates it.
- `AppSettings.validate()` **is incomplete.** `model` is checked only for non-emptiness rather than against `SUPPORTED_MODELS`, and window dimensions, the output directory, and recent-file paths are not validated at all.
- **Secret redaction operates on the formatted message.** `SecretRedactionFilter` redacts `record.getMessage()` only, so a token passed through `record.args` could escape if formatting fails.
- **Test-quality details.** A handful of assertions check only truthiness (`assert numbered`, `assert tables`) or a coarse byte floor (`assert len(data) > 500`). Five tests build objects through `__new__` and set attributes by hand, which is what made three of them break on a constructor change during this review. `tests/test_multi_source.py` imports a helper from `tests/test_transcript_cache.py`, coupling the two files.

Suggested order of work

1. `speechbrain` in `pyproject.toml` — a one-line fix for a broken install. 2. Atomic settings writes with logging on corrupt load. 3. CSV sanitisation and subtitle escaping — small, self-contained, testable. 4. A CI workflow that runs the existing suite, then ruff. 5. Complete `closeEvent`, and move the GPU and Ollama polling off the GUI thread. 6. Decide what to do about the committed meeting PDF.

The remaining items — extracting controllers from `main_window.py`, caching models, indexing diarization turns — are larger and benefit from having CI in place first.